

0.97.1. Une lettre qui tire — A lance plusieurs tirs

Le 0.97, avec les trois tirs du 0.96.1 : garde le bouton A enfoncé, une rafale part, un tir toutes les 250 ms au plus, jamais plus de trois en vol.

1. Ce qu'il faut comprendre

C'est le 0.97, avec plusieurs tirs. Au 0.97, un seul tir à la fois : tant qu'il volait, le bouton A ne faisait rien. Ici, garde A enfoncé : une **rafale** part, jusqu'à **trois** tirs en vol.

Ce qui vient du 0.96.1 : les deux tableaux `tx[3]` et `vole[3]`, la boucle qui lance le premier tir libre (avec `lance`), et la boucle qui fait avancer chaque tir.

Ce qui est nouveau ici : un rythme pour tirer, `chaque(250)`. Au 0.97, `vole == 0` empêchait un deuxième tir. Maintenant, un deuxième tir est permis... et A enfoncé en lancerait un à **chaque image** : trois tirs collés les uns aux autres. On lance donc dans `if (chaque(250)) { if (bouton(A)) { ... } }` : au plus **4 tirs par seconde**, bien espacés.

Les trois volent déjà ? La boucle ne trouve aucun `vole[i]` à 0 : rien ne part. Il faut attendre qu'un tir sorte de l'écran.

Déroulons, A tenu : 0 ms, le tir 0 part de (3, 8). 250 ms : il est en (5, 8), le tir 1 part. 500 ms : le tir 2 part. 750 ms : les trois volent, rien ne part. Vers 1,7 s, le tir 0 sort : sa case est libre, le suivant part.

Essaie : `chaque(100)` : les tirs partent collés, à une case d'écart. Puis `chaque(500)` : une rafale lente.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <chaque>` → [0.29.3. La fonction chaque\(\) — toutes les n millisecondes](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <effacer>` → [0.16.3. La fonction effacer\(\) — effacer des cases](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **657 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
démarrage          52 o ← la console, à l'allumage — la mise en route de la console
chaque_minuteur()  124 o ← main() — une fonction fournie par le langage
main()             260 o ← démarrage — ta fonction
VBlank              8 o  ← la console, à chaque image — compte les images, 60 fois par seconde
```

AttendreImage	28 o	← image()	– attend l'image suivante
AttendreVBlank	16 o	← démarrage	– attend que l'écran soit libre
AttendreAcces	12 o	← EffacerCases, EcrireTexte	– attend de pouvoir écrire une case
AdresseCase	10 o	← texte(), effacer()	– trouve l'adresse d'une case de l'écran
EffacerCases	12 o	← effacer()	– efface des cases
EcrireTexte	10 o	← texte()	– écrit un texte, lettre par lettre
CopierTuiles	40 o	← démarrage	– copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage	– vide les deux cartes de l'écran
LireManette	37 o	← bouton()	– lit les huit boutons
Tuiles	32 o	← CopierTuiles	– 2 lettres sur 43
"."	1 o	← texte()	– un texte gravé
"."	1 o	← texte()	– un texte gravé

Lettres gravées : A.

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, DonneesTransfert, Notes

Fonctions du langage appelées :

texte()	x 2	48 o sur place, dans main()	+ AdresseCase (10 o), EcrireTexte (10 o)
effacer()	x 1	21 o sur place, dans main()	+ AdresseCase (10 o), EffacerCases (12 o)
poser()	x 1	6 o sur place, dans main()	
bouton()	x 1	5 o sur place, dans main()	+ LireManette (37 o)
image()	x 1	3 o sur place, dans main()	+ AttendreImage (28 o)
images()	x 1	2 o sur place, dans chaque_minuteur()	

Tes fonctions appelées :

chaque_minuteur()	x 2	21 o sur place, dans main()
-------------------	-----	-----------------------------

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- UNE LETTRE QUI TIRE : A lance le tir – plusieurs tirs ----
// Le 0.97, mais avec trois tirs (le 0.96.1) : garde le bouton A enfoncé,
// une rafale part, un tir toutes les 250 ms.
```

```
#include <poser>           // pose une tuile sur une case du fond
#include <ALPHABET>        // les lettres de la police : ALPHABET[0] est le A
#include <chaque>          // répond 1 toutes les n millisecondes
#include <bouton>          // lit un bouton de la manette
#include <texte>           // écrit un texte à l'écran
#include <effacer>        // efface des cases, ou tout le fond
```

2. UN TABLEAU — TX

```
uint8_t tx[3];           // les colonnes des trois tirs (le 0.96.1)
```

3. UN TABLEAU — VOLE

```
uint8_t vole[3];         // vole[i] = 1 : le tir n° i est en vol
```

4. UNE VARIABLE — LANCE

```
uint8_t lance = 0;       // 1 = un tir vient de partir, à ce tour-ci
```

5. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    poser(2, 8, ALPHABET[0]); // la lettre A, celle qui tire : (2, 8)

    while (true) {
        image();

        // ---- NOUVEAU : le bouton A lance un tir, au plus toutes les 250 ms ----
        // Sans ce rythme, garder A enfoncé lancerait les trois tirs en trois
        // images, collés les uns aux autres.
        if (chaque(250)) {
            if (bouton(A)) {
                lance = 0;
                for (uint8_t i = 0; i < 3; i++) {
                    if (lance == 0 && vole[i] == 0) { // le premier tir LIBRE part
                        vole[i] = 1;
                        tx[i] = 3;
                        texte(tx[i], 8, ".");
                        lance = 1;
                    }
                }
            }
            // Les trois volent déjà ? Rien ne part : il faut qu'un sorte.
        }

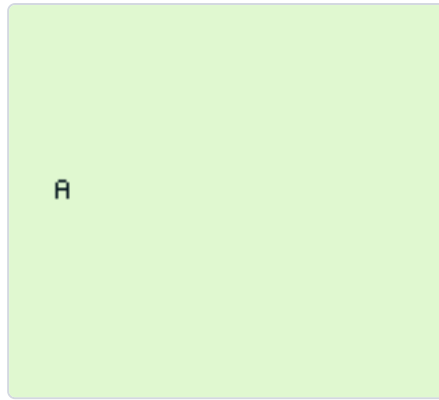
        // ---- chaque tir avance (le 0.96.1) ----
        if (chaque(100)) {
            for (uint8_t i = 0; i < 3; i++) {
                if (vole[i] == 1) {
                    effacer(tx[i], 8, 1);
                    tx[i] = tx[i] + 1;
                    if (tx[i] < 20) {
                        texte(tx[i], 8, ".");
                    } else {
                        vole[i] = 0;
                    }
                }
            }
        }
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



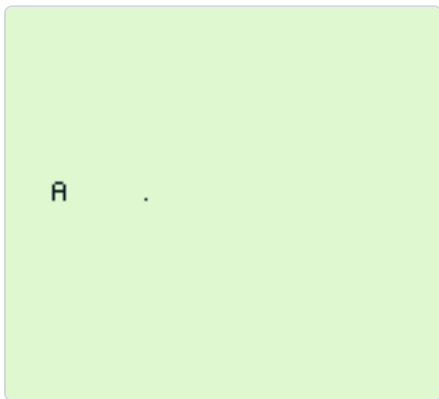
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



A — l'écran a changé

5. Ce qu'on doit voir

La lettre A en (2, 8). Bouton A tenu : une rafale de « . », espacés, jusqu'à trois en vol en même temps ; relâché, les tirs finissent leur course et disparaissent.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ au départ, la lettre A seule
- ✓ A tenu : trois tirs en vol en même temps, pas plus (au plus 3)
- ✓ les tirs partent espacés, jamais collés
- ✓ relâché : tous disparaissent, il ne reste que le A

Ensuite : **0.98. Une lettre qui tire — la place du A dans deux variables** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++